

Cloud handoff — #493 + #494, the stash executor's two contradictions

Written: 2026-08-25 · **By:** max (CLI session on Tom's box) · **For:** a cloud Claude Code session on `tom2025b/git-vista`

These two issues are one decision. Do not take only one of them. #493 asks whether `PopStash` should exist at all; #494 reports that the *sibling* of its executor does not do the thing `PopStash`'s executor documents as essential. Answer #493 first and #494 either collapses into it or becomes trivial.

```
task_id: gv-stash-executor-decision
issues: [493, 494]
milestone: M3 – Parallel Work & Recovery [V2]    # both are #77 follow
repo: tom2025b/git-vista
base: main
branch: fix/m3.24-stash-executor
sign_commits_as:
  name: Claude_Max
  email: 262510778+tom2025b@users.noreply.github.com
sign_artifacts_as: max
adr_number: 0078          # ASSIGNED. Do not pick "the next free" c
allowed_paths:
  - crates/git-vista-protocol/src/plan.rs
  - crates/git-vista-protocol/tests/plan_golden.rs          # constru
  - crates/git-vista-server/src/planner.rs
  - crates/git-vista-server/src/planner/stash.rs
  - crates/git-vista-server/src/planner/contract_suite.rs # exhaust
  - crates/git-vista-mcp/src/plan_tools.rs                  # a separ
  - crates/git-vista-server/src/sandbox/**
  - crates/git-vista-server/src/main.rs                      # only if you wire t
  - crates/git-vista-server/src/route_authz.rs # ditto
  - docs/adr/
forbidden_paths:
  - design-docs/          # untracked; not in your clone
  - crates/git-vista/src/** # the frontend is not yours this re
  - ci/browser/**          # see "What you cannot run" below
  - handoff.md
merge_order: after CLOUD-1 (#495). See "Merge order" below.
```

The decision, stated plainly

`crates/git-vista-protocol/src/plan.rs` says two incompatible things forty-six lines apart: `PopStash { entry, expected_oid }` exists as a live variant at line 1175, and line 1221 says `// PopStash is deliberately ABSENT (M3.24, decided 2026-08-18)`. The variant is fully wired — planner dispatch, executor, risk level, precondition, recovery strategy, sandbox argv, network need — and completely unreachable, because `main.rs` routes no `/api/stash/pop`.

Meanwhile the frontend (merged in PR #490) composes a pop out of apply → read conflicts → drop, in `crates/git-vista/src/features/stash/signals.rs`, with `core::drop_gate` as the single place that decides whether the destructive half runs.

So there are two coherent end states, and one incoherent one (today's):

Option A — delete `PopStash`. The composed pop is the real one, it is host-tested in `features/stash/core.rs`, and the comment at `plan.rs:1221` becomes true. You would remove the variant, its executor, its planner arm, its sandbox entries and its risk/recovery rows. The cost: `exec_pop_stash`'s conflict re-read disappears, so #494's asymmetry must be resolved by *adding* the re-read to `exec_apply_stash` — which is exactly what #494 asks for anyway.

Option B — wire `/api/stash/pop`. The variant becomes reachable, one operation row covers a pop, and the frontend could later stop composing. The cost: `main.rs` carries a written argument for why that route deliberately does not exist ("pop is apply-then-drop and one operation row..."). **Read that comment in full before choosing B.** Overturning a written decision is allowed; doing it without addressing its argument is not.

That argument is stronger than the one-line comment

shows. docs/superpowers/specs/m3.24-stash.md §5 — and issue #493's own body, which quotes it — say PopStash must not ship until one of two prerequisites lands: composite outcomes persisted on the operation row (apply_failed / applied_drop_failed / completed , with a Recovery Centre rendering for each), or PopStash rebuilt as durable orchestration over linked child ApplyStash/DropStash records. ADR 0077's own "Alternatives considered" section, written today, reaches the same conclusion and adds: "Neither exists." Both live in durable.rs , operations.rs , and/or recovery_center.rs — none of which are in allowed_paths . Treat B as needing an allowed_paths amendment and a separate design conversation before it is real work, not as a same-branch alternative to A.

Recommend one, in the ADR, with the argument. Tom's standing preference is the thorough path over the quick one, and a decision that leaves the tree self-consistent over one that leaves a second contradiction behind.

#494 on its own terms, if A is chosen

crates/git-vista-server/src/planner/stash.rs :

- exec_pop_stash (lines ~259-262) reads the conflict state in **both** branches, with a comment saying why: *"a pop git called successful while leaving conflicted paths behind is precisely the case this criterion is about."*
- exec_apply_stash (lines ~191-205) branches on the process exit status alone and never asks.

The issue's own measurement says it does not bite on git 2.43.0 for a content conflict (git stash apply exits 1, UU in porcelain).

Do not treat "it does not bite today" as "it is fine." The reason to fix it is that the guarantee is written down in one

executor and absent in its sibling, and a future git — or a different conflict shape — decides which of the two was right. Say so in the commit message rather than claiming a user-visible bug you cannot demonstrate.

If you *can* find a shape where apply exits 0 with conflicts present, that is a much stronger commit message. Worth twenty minutes: try a stash whose only conflict is a delete/modify, and a stash applied with `--index` against a dirty index.

What you cannot run, and what to do instead

The browser leg does not run in a cloud container. The server refuses to start without its strict sandbox tier, and the kernel there reports `landlock_abi=-1`; INV-13 gives no degraded mode. Two sessions hit this independently on 2026-08-25. Installing `bwrap` changes nothing — it is the kernel's missing capability, not the container's.

This task is server-side, so it should not need the browser at all. If you find yourself wanting a browser assertion, that is a signal the change has reached the frontend, which is outside `allowed_paths` — stop and say so in the PR.

`cargo test -p git-vista-server` is yours and must be green. Note that two tests in that crate —

`recovery_center::tests::a_stale_claimed_undo_is_refused_and_the_branch_is_left_alone` and

`state::tests::selection_flow_carries_mode_and_gates_writes` — flake under parallel execution because they race on the process-global current repository (#438). If one of those is your only red, re-run before believing it; if both of yours are green and only those flake, say so in the PR rather than chasing it.

Acceptance

1. `plan.rs` no longer contains two contradictory statements about `PopStash`. Whichever option you take, the comment and the code agree afterwards.
2. Whatever the executor for "apply a stash" is called at the end, it reads the conflict state on the success path as well as the failure path — the guarantee `exec_pop_stash` currently documents alone. **Do not copy `exec_pop_stash`'s status code for the new success-but-conflicted case.** That path returns `409 CONFLICT`, but the frontend's `apply_stash_request` (`crates/git-vista/src/api/stash.rs`, forbidden to you) derives `ApplyOutcome::Applied` from `resp.ok()` alone — any non-2xx becomes `ApplyOutcome::Refused`. ADR 0077 D6 depends on telling those apart: `Conflicted { apply_refusal: None }` ("applied but left conflicts") versus `Some(..)` ("applying hit conflicts") are deliberately different sentences, and D6 calls the wrong one on the wrong case a false claim. A 409 on git-succeeded-but-left-conflicts would collapse that distinction through a file you cannot fix. Keep the success path's status in the 2xx range and name the conflicted paths in the response body instead.
3. **ADR 0078** records the choice, its alternative, and why the 2026-08-18 decision was kept or overturned.
`docs/adr/README.md` index updated.
4. Every mutation-sensitive test you add is proved able to go red **two different ways** — remove the mechanism, and weaken it. One `caught` verdict is not proof; a Git-Vista test survived one mutation and caught another on 2026-08-22, and either alone gives the wrong answer.
5. `cargo fmt --all`, `cargo clippy --all-targets -- -D warnings`, and `cargo test --workspace` green.
5. PR body says `Closes #493` and `Closes #494`. **Never delete the branch.**

Merge order

Land **after** CLOUD-1 (#495, shared stash DTOs). #495 rewrites field names across `handlers/stash.rs` and the protocol crate; if this PR lands first, #495 pays for the rebase, and #495 is the one whose diff is mechanical enough that a rebase is genuinely risky to review. If you are ready first, say so in the PR and wait rather than merging.

Signed: max · 2026-08-25T10:05:00-04:00